

Midori 算法抗故障攻击安全性评估*

王艺迪¹, 赵新杰^{2,3}, 张帆^{4,5}, 郭世泽^{1,2}, 吴礼发¹, 李文⁶, 楼潇轩⁴

1. 中国人民解放军理工大学, 南京 210007
2. 解放军外国语学院 指挥系, 洛阳 471003
3. 北方电子设备研究所, 北京 100083
4. 浙江大学 信息与电子工程学院, 杭州 310027
5. 保密通信重点实验室, 成都 610041
6. 北京科技大学 计算机与通信工程学院, 北京 100083

通讯作者: 张帆, E-mail: fanzhang@zju.edu.cn

摘要: Midori 是 ASIACRYPT 2015 上提出的一种轻量级分组密码算法, 密钥长度为 128-bit, 分组长度为 64/128-bit, 分别对应 Midori64 和 Midori128, 可被用于保护物联网设备安全. 对 Midori 算法抗故障分析安全性进行了评估. 首先, 基于信息论通过分析故障传播路径, 对故障注入后的 Midori 密钥剩余熵进行了理论估计. 结果表明: 基于第 $R-3$ 轮半字节和字节模型, 1 次故障注入可分别将 Midori64、Midori128 密钥剩余熵大约降低到 68.47-bit、8.03-bit, 但对倒数第 2 轮、第 3 轮故障分析复杂度较高, 多次故障注入分析可解决该问题. 然后, 利用差分故障分析方法, 对故障注入后的 Midori 密钥剩余熵进行了实际验证. 结果表明: 3 次随机半字节、2 次随机字节故障可分别将 Midori64、Midori128 的密钥剩余熵降低至 8.10-bit 和 0-bit. 最后, 利用 Midori 代数方程简单特点, 将代数分析引入到故障分析中, 利用代数故障分析方法优化了 Midori 差分故障分析结果. 结果表明: 代数故障分析可将 Midori64 故障攻击扩展到复杂故障模型, 基于第 $R-3$ 轮字节故障模型、 $R-4$ 轮半字节故障模型, 可分别使用 4 次、10 次故障注入恢复 Midori64 完整密钥; 代数故障分析可以降低 Midori128 攻击复杂度, 基于第 $R-3$ 轮字节故障模型, 1 次故障注入在 94% 的情况下可将 Midori128 密钥熵降低至 16-bit 以内. 因此, 必须对 Midori 算法倒数 5 轮进行故障攻击防护.

关键词: 轻量级分组密码; Midori; 差分故障分析; 代数故障分析; 安全性评估

中图法分类号: TP309.7 **文献标识码:** A **DOI:** 10.13868/j.cnki.jcr.000163

中文引用格式: 王艺迪, 赵新杰, 张帆, 郭世泽, 吴礼发, 李文, 楼潇轩. Midori 算法抗故障攻击安全性评估[J]. 密码学报, 2017, 4(1): 58-78.

英文引用格式: WANG Y D, ZHAO X J, ZHANG F, GUO S Z, WU L F, LI W, LOU X X. Security evaluation for fault attacks on lightweight block cipher Midori[J]. Journal of Cryptologic Research, 2017, 4(1): 58-78.

Security Evaluation for Fault Attacks on Lightweight Block Cipher Midori

WANG Yi-Di¹, ZHAO Xin-Jie^{2,3}, ZHANG Fan^{4,5}, GUO Shi-Ze^{1,2}, WU Li-Fa¹, LI Wen⁶, LOU Xiao-Xuan⁴

1. PLA University of Science and Technology, Nanjing 210007, China
2. Dept. of Command, PLA University of Foreign Language, Luoyang 471003, China
3. The Institute of North Electronic Equipment, Beijing 100083, China

* 基金项目: 国家自然科学基金(61173191, 61272491, 61309021, 61472357, 61571063); 浙江大学青年科研创新专项基金(2015QNA5005); 保密通信重点实验室(9140C110602150C11053)

收稿日期: 2016-10-13 定稿日期: 2016-10-25

4. College of Information Science & Electronic Engineering, Zhejiang University, Hangzhou 310027, China

5. Science and Technology on Communication Security Laboratory, Chengdu 610041, China

6. School of Computer & Communication Engineering, University of Science & Technology, Beijing 100083, China

Corresponding author: ZHAGN Fan, E-mail: fanzhang@zju.edu.cn

Abstract: Midori is a lightweight block cipher of 128-bit key size proposed at ASIACRYPT 2015. It is a family of two block ciphers: Midori64 and Midori128 with 64-bit and 128-bit block size respectively, they can be used to protect small computing devices in IoT. The resistance of Midori64 and Midori128 against fault attacks is evaluated. Firstly, the remained key entropy of Midori is evaluated by analyzing the fault propagation path based on information theory. Theoretical analysis results show that: based on half byte and full byte fault model in round $R-3$, one fault injection can reduce the key entropy of Midori64 and Midori128 to 68.47 and 8.03 bits, respectively. However, the computation complexity in analyzing faults in round $R-2$ to round $R-3$ is unaffordable and multiple fault injections can solve this problem. Then, the remained key entropy of Midori is verified by differential fault analysis (DFA) technique. The result demonstrates that: three random half byte fault injections can reduce the key entropy of Midori64 to 8.10 bits and two random byte fault injections can recover the full secret key of Midori128. Finally, an algebraic technique is introduced into fault attack on Midori and an algebraic fault analysis (AFA) is applied to optimize the DFA result. The results show that: AFA can extend fault attack on Midori64 to more complicated fault models. Based on the byte fault model in round $R-3$ and half byte fault model in round $R-4$, four and ten fault injections can recover the full 128-bit key of Midori64, respectively. Based on the byte fault model in round $R-3$, single fault injection can reduce the key entropy of Midori128 to less than 16-bit for 94% of the cases. Thus, the last 5 rounds of Midori should be protected against fault attacks.

Key words: lightweight block cipher; Midori; differential fault analysis; algebraic fault analysis; security evaluation

1 引言

物联网(The Internet of Things, IoT)是一种建立在互联网上的泛在网络,主要通过 RFID、红外感应器、全球定位系统、激光扫描器等信息传感设备,将任何物品与互联网连接起来,实现物与物、物与人、物与网络的信息交换和通讯,达到万物的智能化识别、定位、跟踪、监控和管理。作为新一代信息化浪潮的典型代表,物联网被称为继计算机、互联网之后世界信息产业发展的第三次浪潮。为了确保信息不会被窃取、篡改、伪造、抵赖,物联网安全需要密码技术的支撑^[1,2]。由于物联网上使用的 RFID、传感器等芯片不同于传统的台式机和高性能计算机,其计算能力、存储资源、功率消耗受限,难以运行传统的重量级密码算法,给安全带来了极大的挑战。即使将传统密码算法优化实现在物联网芯片中,功率消耗、运行速度也相对较差,实现效率不高。在这种情况下,轻量级密码算法应运而生,旨在安全性、面积功耗、时延速度等多个方面实现较好的折中,欧美相继开展了轻量级密码算法的征集计划,研究者相继提出了多个轻量级分组密码、序列密码、公钥密码、哈希算法、认证加密、密码协议。

在轻量级分组密码方面,典型的算法包括 PRESENT^[3]、MIBS^[4]、GOST^[5]、KLEIN^[6]、LED^[7]、Piccolo^[8]、LBlock^[9]、PRINCE^[10]、Midori^[11]等,这些算法大都可 在 3000 个门电路内硬件实现,特别适用于物联网芯片加解密需要。Midori^[11]是 ASIACRYPT 2015 会议上提出的一种轻量级密码,由 Denmark 大学、索尼公司、Lugano 大学的多位知名密码学者设计,分组长度可为 64-bit 和 128-bit,密钥长度为 128-bit,对应的算法分别记为 Midori64 和 Midori128。相比已有轻量级分组密码算法,Midori 的 S 盒和列混淆函数都是符合自逆特性的,4-bit 的 S 盒具有非常小的延迟,相比 PRINCE 和 PRESENT 算法的 S 盒要快 1.2 到 2 倍,轮常量是从圆周率中提取出的随机二进制向量,密钥扩展也相对比较简单,而且轮数也相对较低。总的来说,加解密硬件面积开销和功耗开销较低,特别适用于物联网芯片应用;同时,由于 Midori 算法的密钥长度为 128 位,又可以同时提供较好的安全性。

密码故障分析^[12]是指在密码模块执行过程中主动地利用电源电压毛刺、时钟毛刺、激光干扰、外界温度改变等方式迫使密码模块产生运算错误,并通过对错误运算结果的分析来实现秘密信息恢复。由于物联网芯片大都基于电子技术进行实现,计算、存储资源受限,软硬件安全防护能力有限,同时接口也相对比较简单,较易受到外界条件的干扰,容易被注入故障。近年来,随着故障注入精度以及软硬件逆向技术的提高,实施故障攻击所依赖的前提条件不断减弱,成本不断下降,故障类型和位置已经在一定程度上可以控制,从而使故障攻击逐步成为攻击轻量级分组密码系统的较容易实施且最难防御的攻击技术。差分故障分析(Differential Fault Analysis: DFA)是最早通过引发计算错误来攻击分组密码的技术,由 Biham 和 Shamir 等在 1997 年提出^[13],并对 DES 分组密码进行了分析,之后研究者将该方法推广至 3DES^[14]、AES^[15]、ARIA^[16]、Camellia^[17]、SMS4^[18]等分组密码算法。近年来,随着代数求解技术的发展,代数分析技术逐渐被引入到故障分析中,代数故障分析方法已经成为攻击轻量级分组密码算法的新型技术,研究者相继对 DES^[19]、LED^[20]、Piccolo^[21]等密码算法进行了分析应用。目前,Midori 算法的安全性分析主要集中在传统密码分析,如中间相遇分析^[22]、不变子空间分析^[23]、不可能差分分析^[24]、非线性不变攻击^[25]。最佳的结果表明:在部分弱密钥的情况下,Midori64 是不安全的,Midori128 仍相对比较安全。在抗故障攻击安全性评估方面,国内外尚未发现有公开发表的结果。研究 Midori 算法的安全性,提出有效的故障攻击方法,对于保护物联网的通信安全具有重要意义。

本文在对 Midori 算法组件进行深入分析后,对算法抗故障分析安全性进行了评估。首先,基于信息论通过分析故障传播路径,对故障注入后的 Midori 密钥剩余熵进行了理论估计。结果表明:基于第 $R-3$ 轮半字节和字节模型,1 次故障注入可分别将 Midori64、Midori128 密钥剩余熵大约降低到 68.47-bit、8.03-bit,但对倒数第 2 轮、第 3 轮故障分析复杂度较高,多次故障注入分析可解决该问题。然后,利用差分故障分析方法,对故障注入后的 Midori 密钥剩余熵进行了实际验证。结果表明:3 次随机半字节、2 次随机字节故障可分别将 Midori64、Midori128 的密钥剩余熵降低至 8.10 bit 和 0 bit。最后,利用 Midori 代数方程简单特点,将代数分析引入到故障分析中,利用代数故障分析方法优化了 Midori 差分故障分析结果。结果表明:代数故障分析可将 Midori64 故障攻击扩展到复杂故障模型,基于第 $R-3$ 轮字节故障模型、 $R-4$ 轮半字节故障模型,可分别使用 4 次、10 次故障注入恢复 Midori64 完整密钥;代数故障分析可以降低 Midori128 攻击复杂度,基于第 $R-3$ 轮字节故障模型,1 次故障注入在 94% 的情况下可将 Midori128 密钥剩余熵降低至 16 bit 以内。因此,必须至少对 Midori 算法倒数 5 轮进行故障攻击防护。

本文结构如下:第 2 节介绍 Midori 算法;第 3 节介绍 Midori 算法故障模型、故障传播过程分析和密钥分析原理;第 4 节介绍故障注入后的 Midori 密钥剩余熵理论估计和复杂度分析;第 5 节给出了攻击实验的设计与环境配置;第 6、7 节分析给出了 Midori 差分故障分析、代数故障分析的实施步骤和实验结果分析;第 8 节总结全文。

2 Midori 算法

2.1 符号及术语说明

X, K, Y, Y^*, S : 加密明文、主密钥、正确密文、错误密文、轮密钥加密中间态;

R : 迭代轮数;

$a \leftarrow b$: 将 b 的赋值给 a ;

Z_2^e : e bit 的向量集;

Z_2^4 : 表示半字节;

算法中使用的基本运算有异或($\oplus$), 乘法($\cdot$)和连接($\parallel$)。

2.2 算法描述

Midori 算法^[1]密钥长度为 128-bit, 分组长度可为 64-bit 和 128-bit, 对应的基本运算单元(称为 cell)分别为半字节(4-bit)和字节(8-bit), 对应算法分别记为 Midori64 和 Midori128, 迭代轮数为 16 轮和 20 轮. Midori64 加密流程如图 1 所示. Midori128 的加密流程与图 1 相似, 差别在于迭代轮数不同.

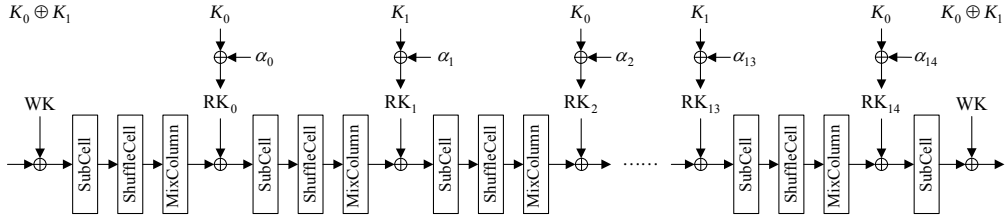


图 1 Midori64 的加密流程

Figure 1 Overview of the encryption of Midori64

(1) SubCell(SC): 采用两个 S 盒 Sb_0, Sb_1 . Midori64 采用 1 个 4-bit 的 S 盒 Sb_0 计算; Midori128 采用四个 8-bit 的 S 盒 $SSb_0, SSb_1, SSb_2, SSb_3$ 计算, 每个 S 盒由两个 Sb_1 计算得出, Sb_0 和 Sb_1 如图 2 所示.

x	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
$Sb_0[x]$	c	a	d	3	e	b	f	7	8	9	1	5	0	2	4	6
$Sb_1[x]$	1	0	5	3	e	2	f	7	d	a	9	b	c	8	4	6

图 2 Midori64 和 Midori128 使用的 4-bit S 盒

Figure 2 4-bit S-box of Midori64 and Midori128

Midori128 中 SSb_i 的计算过程如图 3 所示:

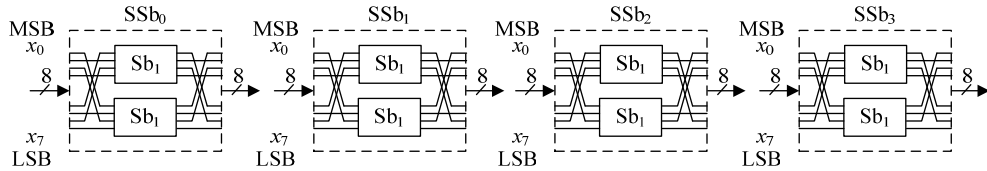


图 3 Midori128 中 SubCell 过程使用的矩阵变换

Figure 3 Matrix Transformation in the SubCell of Midori128

(2) ShuffleCell(SFC): 将 SubCell 结果分为 16 块 ($S_0, S_1, \dots, S_{15}$) 进行按 cell 置换, 如下所示.

$$\begin{bmatrix} S_0 & S_{14} & S_9 & S_7 \\ S_{10} & S_4 & S_3 & S_{13} \\ S_5 & S_{11} & S_{12} & S_2 \\ S_{15} & S_1 & S_6 & S_8 \end{bmatrix} \leftarrow \begin{bmatrix} S_0 & S_4 & S_8 & S_{12} \\ S_1 & S_5 & S_9 & S_{13} \\ S_2 & S_6 & S_{10} & S_{14} \\ S_3 & S_7 & S_{11} & S_{15} \end{bmatrix} \quad M = \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}$$

(3) MixColumn(MC): 将 ShuffleCell 计算结果乘以矩阵 M , 执行按列混淆变换,

$${}^t(S_i, S_{i+1}, S_{i+2}, S_{i+3}) \leftarrow M \cdot {}^t(S_i, S_{i+1}, S_{i+2}, S_{i+3})$$

其中 $i = 0, 2, 4, 8$.

(4) KeyAdd(AK): 将 MixColumn 结果和密钥进行按比特异或计算.

Midori 密钥扩展设计非常简单. 主密钥由两个 64-bit 的密钥 K_0 和 K_1 构成, 主密钥 $K = K_0 \parallel K_1$. Midori64 中, 白化密钥 $WK = K_0 \oplus K_1$, 轮密钥 $RK_i = K_{i \bmod 2} \oplus \alpha_i$ ($0 \leq i \leq 14$), α_i 为轮常量; Midori128 中, 白化密钥 $WK = K$, 轮密钥 $RK_i = K \oplus \beta_i$ ($0 \leq i \leq 18$). 轮密钥中, $\alpha_i = \beta_i$ ($0 \leq i \leq 14$), β_i 为轮常量.

Midori 算法中, 设明文输入和密文分别为 $X = (X_0, X_1, \dots, X_{15})$, $Y = (Y_0, Y_1, \dots, Y_{15})$, 首轮和末轮白化密钥为 WK , 轮密钥为 RK_i . Midori 加解密算法和密钥扩展算法如下所示.

算法 1 Midori 加解密算法 MidoriCore_(R)(Midori64 中 $R=16$, Midori128 中 $R=20$)

$$\text{MidoriCore}_{(R)} : \left\{ \begin{array}{l} \{0,1\}^{16m} \times \{0,1\}^{16m} \times \{0,1\}^{16m} \xrightarrow{R-1} \{0,1\}^{16m} \\ (X, WK, RK_0, \dots, RK_{R-2}) \mapsto Y \end{array} \right.$$

$$\text{MidoriCore}_{(R)}^{-1} : \left\{ \begin{array}{l} \{0,1\}^{16m} \times \{0,1\}^{16m} \times \{0,1\}^{16m} \xrightarrow{R-1} \{0,1\}^{16m} \\ (Y, WK, RK_{R-2}, \dots, RK_0) \mapsto X \end{array} \right.$$

Algorithm MidoriCore_(R)($X, WK, RK_0, \dots, RK_{R-2}$):

$S \leftarrow \text{KeyAdd}(X, WK)$

for $i = 0$ to $R-2$ do

$S \leftarrow \text{SubCell}(S)$

$S \leftarrow \text{ShuffleCell}(S)$

$S \leftarrow \text{MixColumn}(S)$

$S \leftarrow \text{KeyAdd}(S, RK_i)$

$S \leftarrow \text{SubCell}(S)$

$Y \leftarrow \text{KeyAdd}(S, WK)$

Algorithm MidoriCore_(R)⁻¹($Y, WK, RK_{R-2}, \dots, RK_0$):

$S \leftarrow \text{KeyAdd}(Y, WK)$

for $i = (R-2)$ to 0 do

$S \leftarrow \text{SubCell}(S)$

$S \leftarrow \text{MixColumn}(S)$

$S \leftarrow \text{InvShuffleCell}(S)$

$S \leftarrow \text{KeyAdd}(S, L^{-1}(RK_i))$

$S \leftarrow \text{SubCell}(S)$

$X \leftarrow \text{KeyAdd}(S, WK)$

(a) 加密算法

(a) Encryption algorithm

(b) 解密算法

(b) Decryption algorithm

算法 2 Midori 的密钥扩展算法 $L_{(R)}$ 与 $L_{(R)}^{-1}$

(1) Midori64 加密过程密钥扩展算法 $L_{(R)}$ 与解密过程密钥扩展算法 $L_{(R)}^{-1}$ ($R=16$)

$$L_{(RK_i)} : \left\{ \begin{array}{l} \{0,1\}^{128} \times \{0,1\}^{16} \rightarrow \{0,1\}^{128} \\ (K_0, K_1, \beta_0, \beta_1, \dots, \beta_{14}) \mapsto (RK_i) \end{array} \right.$$

$$L_{(RK_i)}^{-1} : \left\{ \begin{array}{l} \{0,1\}^{128} \times \{0,1\}^{16} \rightarrow \{0,1\}^{128} \\ (K_0, K_1, \beta_0, \beta_1, \dots, \beta_{14}) \mapsto (RK_i) \end{array} \right.$$

$L_{(RK_i)}(K_0, K_1, \beta_0, \beta_1, \dots, \beta_{14}) \mapsto (RK_i)$:

$K \leftarrow K_0 \parallel K_1$

for $i = 0$ to $(R-2)$

$RK_i \leftarrow K_{i \bmod 2} \oplus \beta_i$

$WK \leftarrow K_0 \oplus K_1$

$L_{(RK_i)}^{-1}(K_0, K_1, \beta_0, \beta_1, \dots, \beta_{14}) \mapsto (RK_i)$:

$K \leftarrow K_0 \parallel K_1$

for $i = 0$ to $(R-2)$

$RK_i \leftarrow K_{i \bmod 2} \oplus \beta_i$

$RK_i \leftarrow \text{InvShuffleCell}(RK_i)$

$RK_i \leftarrow \text{MixColumn}(RK_i)$

$WK \leftarrow K_0 \oplus K_1$

(a) Midori64 加密过程密钥扩展

(a) Key expansion of Midori64 encryption

(b) Midori64 解密过程密钥扩展

(b) Key expansion of Midori64 decryption

(2) Midori128 加密过程密钥扩展算法 $L_{(R)}$ 与解密过程密钥扩展算法 $L_{(R)}^{-1}$ ($R=20$)

$$\begin{aligned}
& L_{(RK_i)}: \begin{cases} \{0,1\}^{128} \times \{0,1\}^{16} \rightarrow \{0,1\}^{128} \\ (K_0, K_1, \beta_0, \beta_1, \dots, \beta_{18}) \mapsto (RK_i) \end{cases} \\
& L_{(RK_i)}(K_0, K_1, \beta_0, \beta_1, \dots, \beta_{18}) \mapsto (RK_i): \\
& K \leftarrow K_0 \parallel K_1 \\
& \text{for } i = 0 \text{ to } (R-2) \\
& \quad RK_i \leftarrow K \oplus \beta_i \\
& WK \leftarrow K
\end{aligned}$$

(a) Midori128 加密过程密钥扩展
(a) Key expansion of Midori128 encryption

$$\begin{aligned}
& L_{(RK_i)}^{-1}: \begin{cases} \{0,1\}^{128} \times \{0,1\}^{16} \rightarrow \{0,1\}^{128} \\ (K_0, K_1, \beta_0, \beta_1, \dots, \beta_{18}) \mapsto (RK_i) \end{cases} \\
& L_{(RK_i)}^{-1}(K_0, K_1, \beta_0, \beta_1, \dots, \beta_{18}) \mapsto (RK_i): \\
& K \leftarrow K_0 \parallel K_1 \\
& \text{for } i = 0 \text{ to } (R-2) \\
& \quad RK_i \leftarrow K \oplus \beta_i \\
& \quad RK_i \leftarrow \text{InvShuffleCell}(RK_i) \\
& \quad RK_i \leftarrow \text{MixColumn}(RK_i) \\
& WK \leftarrow K
\end{aligned}$$

(b) Midori128 解密过程密钥扩展
(b) Key expansion of Midori128 decryption

3 Midori 故障模型与密钥分析原理

3.1 基本记号与符号

记 X 为明文, $Y, Y^*, \Delta Y$ 分别为正确密文、错误密文和密文差分, K_0, K_1 为初始密钥, WK 为首轮白化和末轮白化密钥, RK_i 为轮密钥; 记 SC^r 为第 $r+1$ 轮 SC 运算输出, SFC^r 为第 $r+1$ 轮 SFC 运算输出, MC^r 为第 $r+1$ 轮 MC 运算输出, AK^r 为第 $r+1$ 轮 AK 运算输出, 其中 $0 \leq r \leq R-1$; 记 ΔSC^r 为第 $r+1$ 轮 SC 输出差分, ΔSFC^r 为第 $r+1$ 轮 SFC 输出差分, ΔMC^r 为第 $r+1$ 轮 MC 输出差分, ΔAK^r 为第 $r+1$ 轮 AK 输出差分; 记 $SC^{-1}, SFC^{-1}, MC^{-1}, AK^{-1}$ 分别为 SC, SFC, MC, AC 运算的逆运算。

3.2 故障模型

本文故障攻击的基本假设为:

- (1) 对同一个明文, 攻击者可以获得在同一个密钥作用下的正确密文 Y 和错误密文 Y^* 。
- (2) 对于 Midori64 来说, 运算的基本单元为一个半字节(4-bit), 攻击者假设可以在算法运行第 13 轮 SubCell 的输出, 即 SC^{12} 注入故障, 使得某一个 cell 出错, 具体的错误值和错误半字节位置未知; 对于 Midori128 来说, 运算的基本单元为一个字节(8-bit), 攻击者假设可以在运算的第 17 轮 SubCell 的输出, 即 SC^{16} 注入故障, 使得某一个 cell 出错, 具体的错误值和错误字节的位置未知。
- (3) 在一次故障攻击中, 密钥的值不发生改变。

3.3 故障传播过程分析

下面以 Midori64、Midori128 在倒数第 4 轮(即第 $R-3$ 轮)密码查表时, 对 SC^{R-4} 的第一个 cell 注入差分值 f 的故障为例, 给出故障传播图, 如图 4 所示。可以看出:

- (1) 经过第 $R-3$ 轮的 MC 函数, 故障被传播到第 2、3、4 个 cell, 差分为 f ;
- (2) 经过第 $R-2$ 轮的 SC 函数, 3 个故障 cell 的位置不变, 但故障差分分别变化为 f_1, f_2, f_3 ; 经过 SFC 函数后, 3 个故障 cell 差分不变, 但故障位置发生了改变; 经过 MC 函数后, 3 个故障 cell 被传播至矩阵 3 列的 9 个 cell, 故障差分值分别变化为 $f_1, f_1, f_1, f_2, f_2, f_2, f_3, f_3, f_3$; 经过 AK 函数, 故障差分 and 位置没发生变化;
- (3) 经过第 $R-1$ 轮的 SC 函数, 9 个故障 cell 位置不变, 但故障差分值分别变化为 $g_1, g_2, g_3, g_4, g_5, g_6, g_7, g_8, g_9$; 经过 SFC 函数后, 故障差分不变, 但故障位置发生了改变; 经过 MC 函数后, 9 个故障 cell 被传播至矩阵 3 列的 16 个 cell, 故障差分变化为 $g_2, g_5, g_9, g_2, g_9, g_5, g_9, g_2, g_5, g_6, g_1, g_6, g_1, g_6, g_1, g_3, g_7, g_3, g_7, g_3, g_7, g_3, g_7, g_4, g_4, g_8, g_4, g_8$; 经过 AK 函数, 故障差分 and 位置没有发生变化;
- (4) 经过第 R 轮的 SC 函数, 故障位置没有改变, 但故障差分分别变化为 $a_0, \dots, a_{15}$; 经过 AK 函数, 故障

差分的位置没有发生变化.

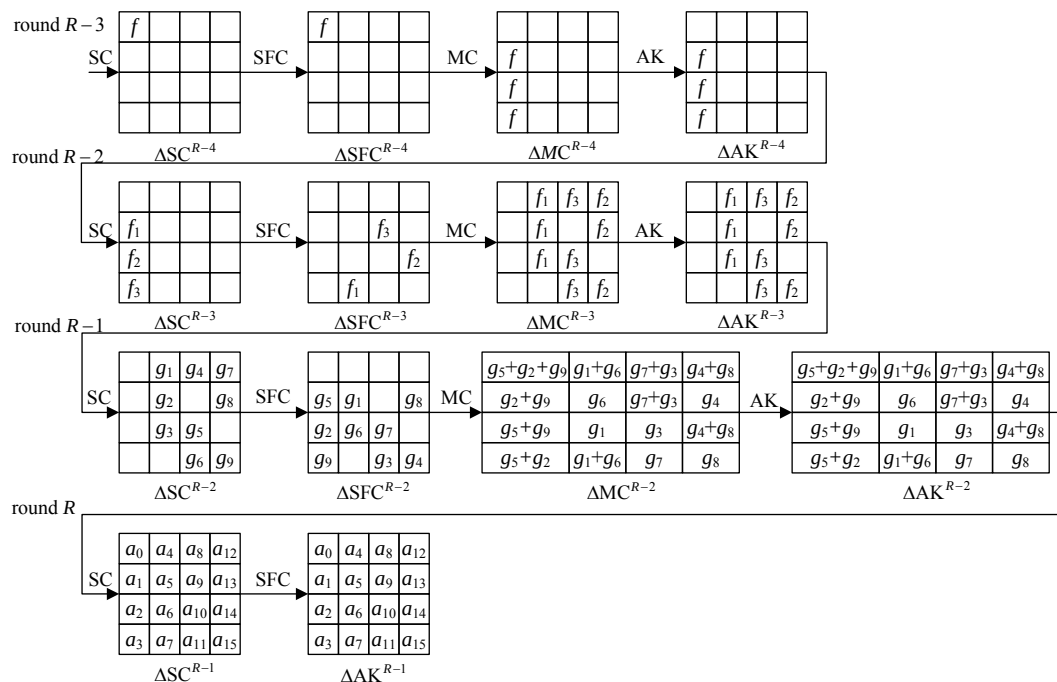


图 4 Midori64 差分故障传播
Figure 4 Differential fault propagation of Midori64

3.4 密钥分析原理

对于 Midori64(Midori128)算法的故障分析原理如下:

- (1) 选择明文 X 进行加密, 获得该明文对应的正确密文 Y ;
- (2) 当加密过程运行到第 $R-3$ 轮时, 对 SC^{R-4} 注入故障, 使得一个 cell 出错, 获得错误密文 Y^* ;
- (3) 结合差分分析, 恢复后白化密钥 WK 的部分信息;
- (4) 重复步骤(1)到(3), 直至完全恢复出 WK 或者将 WK 的搜索空间降低到一个较小范围;
- (5) 根据恢复出的 WK , 对第 R 轮进行解密, 得到第 $R-1$ 轮的正确和错误输出值; 结合差分分析, 恢复第 $R-1$ 轮密钥 RK^{R-2} 值;
- (6) 根据恢复出的 RK^{R-2} 值, 对最后两轮进行解密, 得到第 $R-2$ 轮的正确和错误输出值; 结合差分分析, 恢复第 $R-2$ 轮密钥 RK^{R-3} 的部分值, 并验证 WK 和 RK^{R-2} 的正确性;
- (7) 结合恢复的 WK, RK^{R-2}, RK^{R-3} 值, 恢复主密钥 K .

需要说明的, 上述过程可以通过人工的差分故障分析方法来完成, 也可以利用自动的代数故障分析方法来完成, 此时需要将分析过程转化为代数方程组, 利用解析器来自动的求解密钥.

4 Midori 故障攻击密钥剩余熵与复杂度分析

在第 $R-3$ 轮注入故障, Midori64 和 Midori128 的故障传播图相同. 不同点在于 Midori64 和 Midori128 分别以半字节、字节为基本处理单元. 此外, Midori64 的最后三轮密钥(RK^{13}, RK^{14}, WK)中, 相邻轮密钥没有相关性, 最后一轮对 WK 的分析结果无法降低 RK^{14} 的搜索空间; 而 Midori128 的最后三轮密钥

(RK¹⁷, RK¹⁸, WK) 中, 相邻轮密钥之间可以直接推导, 每轮故障分析结果可以相互利用. 令基本处理单元 cell 的长度为 b , $\varphi(K)$ 表示 K 的剩余信息熵. 下面首先在故障注入位置已知情况下, 分析密钥分析效率.

(1) 分析 WK

通过分析图 4 中故障在第 $R-1$ 轮和第 R 轮的传播过程, 可以看出, 故障传播到最后一轮 SC 运算的 16 个 S 盒, 并发生了故障重叠现象. 显然, 最后一轮 SC 输出每一列的故障差分可以由正确和错误密文计算出来. ΔSC^{R-1} 四列的信息熵分别为 $4b$, $2b + 2\log 2(2^b - 1)$, $2b + 2\log 2(2^b - 1)$, $2b + 2\log 2(2^b - 1)$ -bit, 即

$$\varphi(\Delta SC^{R-1}) = 10b + 6\log 2(2^b - 1) \quad (1)$$

通过穷举 WK 四列的 $4b$ -bit 值, 经过逆 AK、逆 SC、逆 MC 函数后, ΔSFC^{R-2} 的四列差分值的的信息熵分别为 $3\log 2(2^b - 1)$ -bit, $2\log 2(2^b - 1)$ -bit, $2\log 2(2^b - 1)$ -bit, $2\log 2(2^b - 1)$ -bit, 即

$$\varphi(\Delta SFC^{R-2}) = 9\log 2(2^b - 1) \quad (2)$$

由于 S 盒非线性特性, 1 次故障分析后, WK 中间损失的信息熵, 即 WK 的不可能密钥熵基本等价于

$$\varphi(\Delta SC^{R-1}) - \varphi(\Delta SFC^{R-2}) = 10b - 3\log 2(2^b - 1) \quad (3)$$

那么 WK 的密钥剩余熵可表示为

$$\varphi(WK) = 6b + 3\log 2(2^b - 1) \quad (4)$$

攻击的内存计算复杂度为 2^{4b+2} 次密钥穷举, 对于 Midori64、Midori128 来说, 复杂度分别为 2^{18} 和 2^{34} 次穷举. 此外, 1 次故障注入后 WK 四列的密钥剩余熵为 $3\log 2(2^b - 1)$, $2b$, $2b$, $2b$.

(2) 分析 RK^{R-2}

通过分析图 4 中故障在第 $R-2$ 和第 $R-1$ 轮的传播过程, 可以看出, 故障传播到了第 $R-1$ 轮 SC 运算的 9 个 S 盒, 没有发生故障重叠现象. 故障没有被传播到 SC^{R-2} 的第一列, 第 2、3、4 列分别只有 3 个 cell 发生故障, 则 ΔSC^{R-2} 的 4 列的信息熵为 0 , $3\log 2(2^b - 1)$, $3\log 2(2^b - 1)$, $3\log 2(2^b - 1)$.

$$\varphi(\Delta SC^{R-2}) = 9\log 2(2^b - 1) \quad (5)$$

攻击中, SC^{R-2} 的第一列分析无法降低 RK^{R-2} 的密钥熵; 对于第 2、3、4 列, 经过逆 SC、逆 MC 函数后, ΔSFC^{R-3} 的对应列差分值的的信息熵分别为 $\log 2(2^b - 1)$ -bit, $\log 2(2^b - 1)$ -bit, $\log 2(2^b - 1)$ -bit.

$$\varphi(\Delta SFC^{R-3}) = 3\log 2(2^b - 1) \quad (6)$$

穷举 RK^{R-2}、WK 计算 ΔSC^{R-2} , ΔSFC^{R-3} , 1 次故障注入后 RK^{R-2}、WK 的联合信息熵的降低值可表示为

$$\varphi(\Delta SC^{R-2}) - \varphi(\Delta SFC^{R-3}) = 6\log 2(2^b - 1) \quad (7)$$

对于 Midori64 来说, RK^{R-2} 和 WK 是独立的, 1 次故障注入后主密钥 K 的信息熵可被降低为

$$\varphi(K) = 6b + 3\log 2(2^b - 1) + 16b - 6\log 2(2^b - 1) = 22b - 3\log 2(2^b - 1) = 76.28 \quad (8)$$

在 Midori64 的攻击过程中, 1 次故障注入后, 对 SC^{R-2} 的第 2、3、4 列进行分析时, 由于 SFC 移位扩散函数的存在, 攻击者需要对 WK 的 3 列候选值和 RK^{R-2} 的 3 个 cell 进行穷举, 对应穷举复杂度相等, 即

$$(2^b - 1)^3 \times 2^{2b} \times 2^{2b} \times 2^{2b} = (2^b - 1)^3 \times 2^{6b} = 2^{35.72} \quad (9)$$

对于 Midori128 来说, RK^{R-2} 由 WK 异或上一个常量计算得到, SC^{R-2}、 ΔSC^{R-2} 的信息熵和最后一轮分析后 RK^{R-2} 的信息熵等价, 1 次故障注入后主密钥 K 的信息熵可被降低为

$$\varphi(K) = 6b + 3\log 2(2^b - 1) - 6\log 2(2^b - 1) = 6b - 3\log 2(2^b - 1) = 40.02 \quad (10)$$

同样, 1 次故障注入后, 对 SC^{R-2} 的第 2、3、4 列进行分析时, 由于 SFC 移位扩散函数的存在, 攻击者需要对 WK 的 3 列候选值和 RK^{R-2} 的 3 个 cell 进行穷举计算, 分别对应的穷举复杂度为

$$(2^b - 1)^3 \times 2^{2b} \times 2^{2b} = 2^{55.98}, (2^b - 1)^3 \times 2^{2b} \times 2^{2b} \times 2^{3b} = 2^{79.98}, (2^b - 1)^3 \times 2^{2b} \times 2^{2b} = 2^{55.98} \quad (11)$$

实际 Midori64 和 Midori128 攻击中, 为了降低复杂度, 可以进行多次故障注入(如 2 次或 3 次), 首先将 WK 的密钥剩余熵降低到较小值, 然后分析 RK^{R-2} .

(3) 分析 RK^{R-3}

通过分析图 4 中故障在第 $R-3$ 和第 $R-2$ 轮的传播过程, 可以看出, 故障传播到了第 $R-2$ 轮 SC 运算的 3 个 S 盒, 没有发生故障重叠现象. 故障被传播到 SC^{R-3} 的第一列, ΔSC^{R-3} 的信息熵为 $3\log 2(2^b - 1)$ -bit, ΔSFC^{R-4} 的信息熵为 $\log 2(2^b - 1)$ -bit. 1 次故障注入后 RK^{R-3} 、 RK^{R-2} 、WK 的联合信息熵的降低值可表示为

$$\varphi(\Delta SC^{R-3}) - \varphi(\Delta SFC^{R-4}) = 2\log 2(2^b - 1) \quad (12)$$

对于 Midori64 来说, RK^{R-2} 和 WK 是独立的, RK^{R-3} 可以由 RK^{R-2} 和 WK 直接计算出来. 则 1 次故障注入后 $\varphi(K)$ 可被降低为

$$\varphi(K) = 6b + 3\log 2(2^b - 1) + 16b - 6\log 2(2^b - 1) - 2\log 2(2^b - 1) = 22b - 5\log 2(2^b - 1) = 68.47 \quad (13)$$

对于 Midori128 来说, RK^{R-3} 可由 WK 异或上一个常量计算得到, 1 次故障注入后 $\varphi(K)$ 可被降低为

$$\varphi(K) = 6b + 3\log 2(2^b - 1) - 6\log 2(2^b - 1) - 2\log 2(2^b - 1) = 6b - 5\log 2(2^b - 1) = 8.03 \quad (14)$$

令 $\theta^1, \theta^2, \theta^3$ 分别表示倒数三轮白化密钥 WK、轮密钥 RK^{R-2} 、轮密钥 RK^{R-3} . 表 1 给出了已知故障位置时 1 次故障注入后 Midori64 和 Midori128 每个轮次的密钥恢复效率.

表 1 已知故障注入位置, 1 次故障注入后的密钥恢复效率分析
Table 1 Key recovery efficiency of single fault injection under known fault location scenario

密码轮	θ^n 候选值	Midori64 θ^n 候选值	Midori64 $\varphi(K)$	Midori128 θ^n 候选值	Midori128 $\varphi(K)$
R 至 R-1 轮	$2^{6b}/(2^b - 1)^3$	$2^{24}/(15)^3$	$88 + 3\log 2(15) = 99.72$	$2^{48}/(255)^3$	$48 + 3\log 2(255) = 71.98$
R-1 至 R-2 轮	$2^{16b}/(2^b - 1)^6$	$2^{64}/(15)^6$	$88 - 3\log 2(15) = 76.28$	$2^{128}/(255)^6$	$48 - 3\log 2(255) = 24.02$
R-2 至 R-3 轮	$2^{16b}/(2^b - 1)^2$	$2^{64}/(15)^2$	$88 - 5\log 2(15) = 68.47$	$2^{128}/(255)^1$	$48 - 5\log 2(255) = 8.03$

需要说明的是, 当故障注入位置未知时, 可能的故障注入位置一共有 16 种, 则分析得到的密钥剩余熵约等于已知故障位置时计算出的密钥剩余熵加上 4 个比特.

5 攻击实验设计与环境配置

根据第 4 节分析结果可以看出:

(1) 基于 3.2 节故障模型, 对于 Midori64 算法来说, 从信息熵分析的角度, 不论采用什么方法, 至少需要 2 到 3 次故障注入才能有望将密钥搜索空间降低到较小范围;

(2) 对于 Midori128 算法来说, 从信息熵分析的角度, 1 次故障注入有望将密钥搜索空间降低到较小范围. 但是需要说明的是, 利用差分故障分析方法, 第 $R-1, R-2$ 轮分析所需的计算复杂度过高. 为了降低计算复杂度, 一种方法是进行 2 次故障注入, 利用差分故障分析方法, 将最后一轮密钥的搜索空间降低到

较小值, 然后对第 $R-1, R-2$ 轮进行分析, 进而恢复主密钥。

此外, 根据图 3 可知, Midori128 的 S 盒由两个 4-bit 的小 S 盒前后进行一个 8 比特 P 置换得到, S 盒的每个输出比特只和 4 个输入比特相关, 最大阶为 3, 代数复杂度不高。因此, 可以尝试采用代数故障分析方法, 将 Midori128 算法、故障信息、密文信息转化为代数方程组, 利用解析器求解方程恢复密钥。

考虑到上述情况, 设计了 2 种攻击实验, 分别为 Midori 的差分故障分析、Midori 代数故障分析实验。需要说明的是, 鉴于本文主要对故障分析进行研究, 故障注入不是本文的研究重点, 读者可阅读文献 [26,27] 了解故障注入技术, 故文中故障注入均为计算机仿真模拟实现, 使用 C++ 编程语言 (Visual Studio 2010 环境) 通过修改 Midori 加密过程仿真故障注入和分析。PC 机配置为 Intel(R)Core(TM) i7-2640 M、2.80 GHz、4 GB 内存、Windows XP 64 位操作系统。在代数故障分析中, 代数方程求解器采用 CryptoMinisat v 2.9.4^[28]。

6 Midori 算法差分故障分析

6.1 第 R 轮 WK 分析

下面以故障注入到第 $R-3$ 轮第一个 cell 为例给出密钥分析过程。

(1) 攻击中, 对选择明文 X 进行加密, 获得该明文对应的正确密文 Y ; 对于相同明文 X , 在第 $R-3$ 轮注入故障, 得到错误密文 Y^* 。

(2) 计算第 R 轮 SC 函数输出故障差分 ΔSC^{R-1}

$$\Delta SC_i^{R-1} = Y_i \oplus Y_i^* \quad (15)$$

(3) 将 WK 划分为 4 个 4b 比特片段, 分别进行 2^{4b} 次穷举计算第 R 轮 SC 函数的输入故障差分 ΔMC^{R-2} :

$$\begin{aligned} \Delta MC^{R-2} &= \Delta MC_{0,1,2,3}^{R-2} \parallel \Delta MC_{4,5,6,7}^{R-2} \parallel \Delta MC_{8,9,10,11}^{R-2} \parallel \Delta MC_{12,13,14,15}^{R-2} \\ \Delta MC_{0,1,2,3}^{R-2} &= SC^{-1}(WK_{0,1,2,3} \oplus Y_{0,1,2,3}) \oplus SC^{-1}(WK_{0,1,2,3} \oplus Y_{0,1,2,3}^*) \\ \Delta MC_{4,5,6,7}^{R-2} &= SC^{-1}(WK_{4,5,6,7} \oplus Y_{4,5,6,7}) \oplus SC^{-1}(WK_{4,5,6,7} \oplus Y_{4,5,6,7}^*) \\ \Delta MC_{8,9,10,11}^{R-2} &= SC^{-1}(WK_{8,9,10,11} \oplus Y_{8,9,10,11}) \oplus SC^{-1}(WK_{8,9,10,11} \oplus Y_{8,9,10,11}^*) \\ \Delta MC_{12,13,14,15}^{R-2} &= SC^{-1}(WK_{12,13,14,15} \oplus Y_{12,13,14,15}) \oplus SC^{-1}(WK_{12,13,14,15} \oplus Y_{12,13,14,15}^*) \end{aligned} \quad (16)$$

(4) 根据 ΔMC^{R-2} 中不同 cell 的差分值之间的关系, 剔除错误的 WK 候选值。

对于 $WK_{0,1,2,3}$ 的 2^{4b} 个候选值, 正确的候选值应满足:

$$\Delta MC_{0,1,2,3}^{R-2} = g_5 \oplus g_2 \oplus g_9 \parallel g_2 \oplus g_9 \parallel g_5 \oplus g_9 \parallel g_2 \oplus g_5$$

对于 $WK_{4,5,6,7}$ 的 2^{4b} 个候选值, 正确的候选值应满足: $\Delta MC_{4,5,6,7}^{R-2} = g_1 \oplus g_6 \parallel g_6 \parallel g_1 \parallel g_1 \oplus g_6$;

对于 $WK_{8,9,10,11}$ 的 2^{4b} 个候选值, 正确的候选值应满足: $\Delta MC_{8,9,10,11}^{R-2} = g_3 \oplus g_7 \parallel g_3 \oplus g_7 \parallel g_3 \parallel g_7$;

对于 $WK_{12,13,14,15}$ 的 2^{4b} 个候选值, 正确的候选值应满足: $\Delta MC_{12,13,14,15}^{R-2} = g_4 \oplus g_8 \parallel g_4 \oplus g_8 \parallel g_8$ 。

经过分析可得到 WK 的部分候选值

(5) 进行多次故障注入, 重复执行(1)–(4)步骤, 直至得到有限的或者唯一的 WK 候选值。

6.2 第 $R-1$ 轮 RK ^{$R-2$} 分析

(1) 根据恢复的 WK 候选值, 计算出倒数第二轮的正确输出和错误输出值。

(2) 计算倒数第二轮 SC 函数的输出故障差分 ΔSC^{R-2} 。

$$\Delta SC_i^{R-2} = SFC^{-1}(MC^{-1}(SC^{-1}(Y_i \oplus WK_i))) \oplus SFC^{-1}(MC^{-1}(SC^{-1}(Y_i^* \oplus WK_i))) \quad (17)$$

(3) 计算倒数第二轮 SC 函数的输入故障差分 ΔMC^{R-3} 。

$$\begin{aligned}
\Delta MC^{R-3} &= \Delta MC_{0,1,2,3}^{R-3} \parallel \Delta MC_{4,5,6,7}^{R-3} \parallel \Delta MC_{8,9,10,11}^{R-3} \parallel \Delta MC_{12,13,14,15}^{R-3} \\
\Delta MC_{0,1,2,3}^{R-3} &= SC^{-1} \left(RK_{-e_{0,1,2,3}}^{R-2} \oplus SFC^{-1} \left(MC^{-1} \left(SC^{-1} (Y_{0,1,2,3} \oplus WK_{0,1,2,3}) \right) \right) \right) \oplus \\
&\quad SC^{-1} \left(RK_{-e_{0,1,2,3}}^{R-2} \oplus SFC^{-1} \left(MC^{-1} \left(SC^{-1} (Y_{0,1,2,3}^* \oplus WK_{0,1,2,3}) \right) \right) \right) \\
\Delta MC_{4,5,6,7}^{R-3} &= SC^{-1} \left(RK_{-e_{4,5,6,7}}^{R-2} \oplus SFC^{-1} \left(MC^{-1} \left(SC^{-1} (Y_{4,5,6,7} \oplus WK_{4,5,6,7}) \right) \right) \right) \oplus \\
&\quad SC^{-1} \left(RK_{-e_{4,5,6,7}}^{R-2} \oplus SFC^{-1} \left(MC^{-1} \left(SC^{-1} (Y_{4,5,6,7}^* \oplus WK_{4,5,6,7}) \right) \right) \right) \\
\Delta MC_{8,9,10,11}^{R-3} &= SC^{-1} \left(RK_{-e_{8,9,10,11}}^{R-2} \oplus SFC^{-1} \left(MC^{-1} \left(SC^{-1} (Y_{8,9,10,11} \oplus WK_{8,9,10,11}) \right) \right) \right) \oplus \\
&\quad SC^{-1} \left(RK_{-e_{8,9,10,11}}^{R-2} \oplus SFC^{-1} \left(MC^{-1} \left(SC^{-1} (Y_{8,9,10,11}^* \oplus WK_{8,9,10,11}) \right) \right) \right) \\
\Delta MC_{12,13,14,15}^{R-3} &= SC^{-1} \left(RK_{-e_{12,13,14,15}}^{R-2} \oplus SFC^{-1} \left(MC^{-1} \left(SC^{-1} (Y_{12,13,14,15} \oplus WK_{12,13,14,15}) \right) \right) \right) \oplus \\
&\quad SC^{-1} \left(RK_{-e_{12,13,14,15}}^{R-2} \oplus SFC^{-1} \left(MC^{-1} \left(SC^{-1} (Y_{12,13,14,15}^* \oplus WK_{12,13,14,15}) \right) \right) \right)
\end{aligned}$$

其中, RK_{-e}^{R-2} 为 RK^{R-2} 的等价解密密钥

$$RK_{-e}^{R-2} = MC^{-1} \left(SFC^{-1} (RK^{R-2}) \right) \quad (18)$$

(4) 根据 ΔMC^{R-3} 中不同 cell 差分值之间的关系, 剔除错误的等价密钥 RK_{-e}^{R-2} 候选值.

对于 $RK_{-e_{0,1,2,3}}^{R-2}$ 的 2^{4b} 个候选值, 正确的候选值应满足: $\Delta MC_{0,1,2,3}^{R-3} = 0 \parallel 0 \parallel 0 \parallel 0$;

对于 $RK_{-e_{4,5,6,7}}^{R-2}$ 的 2^{4b} 个候选值, 正确的候选值应满足: $\Delta MC_{4,5,6,7}^{R-3} = f_1 \parallel f_1 \parallel f_1 \parallel f_1$;

对于 $RK_{-e_{8,9,10,11}}^{R-2}$ 的 2^{4b} 个候选值, 正确的候选值应满足: $\Delta MC_{8,9,10,11}^{R-3} = f_3 \parallel 0 \parallel f_3 \parallel f_3$;

对于 $RK_{-e_{12,13,14,15}}^{R-2}$ 的 2^{4b} 个候选值, 正确的候选值应满足: $\Delta MC_{12,13,14,15}^{R-3} = f_2 \parallel f_2 \parallel 0 \parallel f_2$.

(5) 重复执行(1)-(4)步骤, 直至得到有限的等价密钥 RK_{-e}^{R-2} 候选值.

(6) 根据 Midori 算法设计, 由等价密钥 RK_{-e}^{R-2} 计算出 RK^{R-2} 候选值

$$RK^{R-2} = SFC \left(MC(RK_{-e}^{R-2}) \right) \quad (19)$$

6.3 第 $R-2$ 轮 RK^{R-3} 分析

由于 RK^{R-2} 可以通过 WK 和 RK^{R-2} 来推导出来, 这部分可以用于验证前面得到的 WK 和 RK^{R-2} 候选值的正确性, 进一步降低主密钥搜索空间.

(1) 根据恢复的每个 WK , RK^{R-2} 候选值, 计算出倒数第三轮正确和错误输出, 以及 RK^{R-3} 的等价解密密钥 RK_{-e}^{R-3} 候选值. 对于 Midori-64 来说, 满足

$$RK_{-e}^{R-3} = MC^{-1} \left(SFC^{-1} (RK^{R-2}) \right) = MC^{-1} \left(SFC^{-1} (WK \oplus RK^{R-2} \oplus \alpha_{R-2} \oplus \alpha_{R-3}) \right) \quad (20)$$

对于 Midori-128 来说, 满足

$$\begin{aligned}
RK_{-e}^{R-3} &= MC^{-1} \left(SFC^{-1} (RK^{R-2}) \right) = MC^{-1} \left(SFC^{-1} (WK \oplus \beta_{R-3}) \right) \\
&= MC^{-1} \left(SFC^{-1} (RK^{R-2} \oplus \beta_{R-2} \oplus \beta_{R-3}) \right)
\end{aligned} \quad (21)$$

(2) 计算倒数第三轮 SC 函数输出故障差分值 ΔSC_i^{R-3}

$$\begin{aligned}
\Delta SC_i^{R-3} &= SFC^{-1} \left(MC^{-1} \left(SC^{-1} \left(RK^{R-2} \oplus \left(SFC^{-1} \left(MC^{-1} \left(SC^{-1} (Y_i \oplus WK_i) \right) \right) \right) \right) \right) \right) \oplus \\
&\quad SFC^{-1} \left(MC^{-1} \left(SC^{-1} \left(RK^{R-2} \oplus \left(SFC^{-1} \left(MC^{-1} \left(SC^{-1} (Y_i^* \oplus WK_i) \right) \right) \right) \right) \right) \right)
\end{aligned} \quad (22)$$

(3) 计算倒数第三轮 SubCell 的输入故障差分 ΔMC^{R-4} .

$$\begin{aligned}\Delta MC^{R-4} &= \Delta MC_{0,1,2,3}^{R-4} \parallel \Delta MC_{4,5,6,7}^{R-4} \parallel \Delta MC_{8,9,10,11}^{R-4} \parallel \Delta MC_{12,13,14,15}^{R-4} \\ \Delta MC_{0,1,2,3}^{R-4} &= SC^{-1} \left(RK_e_{0,1,2,3}^{R-3} \oplus SFC^{-1} \left(MC^{-1} \left(SC^{-1} \left(RK_e_{0,1,2,3}^{R-2} \oplus SFC^{-1} \left(MC^{-1} \left(SC^{-1} (Y_{0,1,2,3} \oplus WK_{0,1,2,3}) \right) \right) \right) \right) \right) \right) \\ &\quad \oplus SC^{-1} \left(RK_e_{0,1,2,3}^{R-3} \oplus SFC^{-1} \left(MC^{-1} \left(SC^{-1} \left(RK_e_{0,1,2,3}^{R-2} \oplus SFC^{-1} \left(MC^{-1} \left(SC^{-1} (Y_{0,1,2,3}^* \oplus WK_{0,1,2,3}) \right) \right) \right) \right) \right) \right) \\ \Delta MC_{4,5,6,7}^{R-4} &= SC^{-1} \left(RK_e_{4,5,6,7}^{R-3} \oplus SFC^{-1} \left(MC^{-1} \left(SC^{-1} \left(RK_e_{4,5,6,7}^{R-2} \oplus SFC^{-1} \left(MC^{-1} \left(SC^{-1} (Y_{4,5,6,7} \oplus WK_{4,5,6,7}) \right) \right) \right) \right) \right) \right) \\ &\quad \oplus SC^{-1} \left(RK_e_{4,5,6,7}^{R-3} \oplus SFC^{-1} \left(MC^{-1} \left(SC^{-1} \left(RK_e_{4,5,6,7}^{R-2} \oplus SFC^{-1} \left(MC^{-1} \left(SC^{-1} (Y_{4,5,6,7}^* \oplus WK_{4,5,6,7}) \right) \right) \right) \right) \right) \right) \\ \Delta MC_{8,9,10,11}^{R-4} &= SC^{-1} \left(RK_e_{8,9,10,11}^{R-3} \oplus SFC^{-1} \left(MC^{-1} \left(SC^{-1} \left(RK_e_{8,9,10,11}^{R-2} \oplus SFC^{-1} \left(MC^{-1} \left(SC^{-1} (Y_{8,9,10,11} \oplus WK_{8,9,10,11}) \right) \right) \right) \right) \right) \right) \\ &\quad \oplus SC^{-1} \left(RK_e_{8,9,10,11}^{R-3} \oplus SFC^{-1} \left(MC^{-1} \left(SC^{-1} \left(RK_e_{8,9,10,11}^{R-2} \oplus SFC^{-1} \left(MC^{-1} \left(SC^{-1} (Y_{8,9,10,11}^* \oplus WK_{8,9,10,11}) \right) \right) \right) \right) \right) \right) \\ \Delta MC_{12,13,14,15}^{R-4} &= SC^{-1} \left(RK_e_{12,13,14,15}^{R-3} \oplus SFC^{-1} \left(MC^{-1} \left(SC^{-1} \left(RK_e_{12,13,14,15}^{R-2} \oplus SFC^{-1} \left(MC^{-1} \left(SC^{-1} (Y_{12,13,14,15} \oplus WK_{12,13,14,15}) \right) \right) \right) \right) \right) \right) \\ &\quad \oplus SC^{-1} \left(RK_e_{12,13,14,15}^{R-3} \oplus SFC^{-1} \left(MC^{-1} \left(SC^{-1} \left(RK_e_{12,13,14,15}^{R-2} \oplus SFC^{-1} \left(MC^{-1} \left(SC^{-1} (Y_{12,13,14,15}^* \oplus WK_{12,13,14,15}) \right) \right) \right) \right) \right) \right)\end{aligned}$$

(4) 根据 ΔMC^{R-4} 中不同 cell 差分之间的关系, 剔除错误的等价密钥 RK_e^{R-3} 候选值, 并相应地剔除对应 WK 和 RK^{R-2} 的候选值.

对于 $RK_e_{0,1,2,3}^{R-3}$ 的 2^{4b} 个候选值, 正确的候选值应满足: $\Delta MC_{0,1,2,3}^{R-3} = 0 \parallel f \parallel f \parallel f$;

对于 $RK_e_{4,5,6,7}^{R-3}$ 的 2^{4b} 个候选值, 正确的候选值应满足: $\Delta MC_{4,5,6,7}^{R-3} = 0 \parallel 0 \parallel 0 \parallel 0$;

对于 $RK_e_{8,9,10,11}^{R-3}$ 的 2^{4b} 个候选值, 正确的候选值应满足: $\Delta MC_{8,9,10,11}^{R-3} = 0 \parallel 0 \parallel 0 \parallel 0$;

对于 $RK_e_{12,13,14,15}^{R-3}$ 的 2^{4b} 个候选值, 正确的候选值应满足: $\Delta MC_{12,13,14,15}^{R-3} = 0 \parallel 0 \parallel 0 \parallel 0$.

(5) 重复执行(1)-(4)步骤, 直至得到有限的 WK, RK^{R-2} 候选值.

(6) 根据得出的白化密钥 WK 以及轮密钥 RK^{R-2} , 恢复主密钥 K_0 和 K_1 .

6.4 Midori64 实验结果与分析

Midori64 算法差分故障攻击中, 首先在已知故障位置下, 基于第 13 轮半字节故障模型, 在不同故障注入次数下执行了攻击实验.

(1) 最后一轮分析

首先对不同故障次数下最后一轮攻击后 WK 的密钥剩余熵进行了统计分析, 每次统计执行 1000 次攻击, 结果如图 5 所示. 可以看出, $N=1$ 时, WK 的密钥剩余熵取值范围为 32 到 50-bit, 均值为 38.35-bit, 接近于理论值 $6b + 3\log 2(2^b - 1) = 35.72\text{-bit}$; $N=2$ 时, 密钥剩余熵的取值范围为 8 到 28-bit, 均值为 15.35-bit; $N=3$ 时, 密钥剩余熵的取值范围为 0 到 12-bit, 均值为 3.46-bit; $N=4$ 时, 密钥剩余熵的取值范围为 0 到 5-bit, 均值为 0.65-bit. 大约 2 到 3 次故障注入即可将 WK 的搜索空间降低到一个较小的值.

(2) 倒数 2 轮、倒数 3 轮分析

在 $N=3$ 时对不同故障次数下最后 2、3 轮攻击后主密钥 K 的剩余熵进行了统计分析, 每次统计执行 1000 次攻击, 结果如图 6 所示. 可以看出, 最后 2 轮攻击后, 主密钥剩余熵的取值范围为 7 到 28-bit, 均值为 15.33-bit; 最后 3 轮攻击后, 主密钥剩余熵的取值范围为 0 到 23-bit, 均值为 8.10-bit. 这样, 3 次故障注入可将主密钥的搜索空间降低到 8.10-bit.

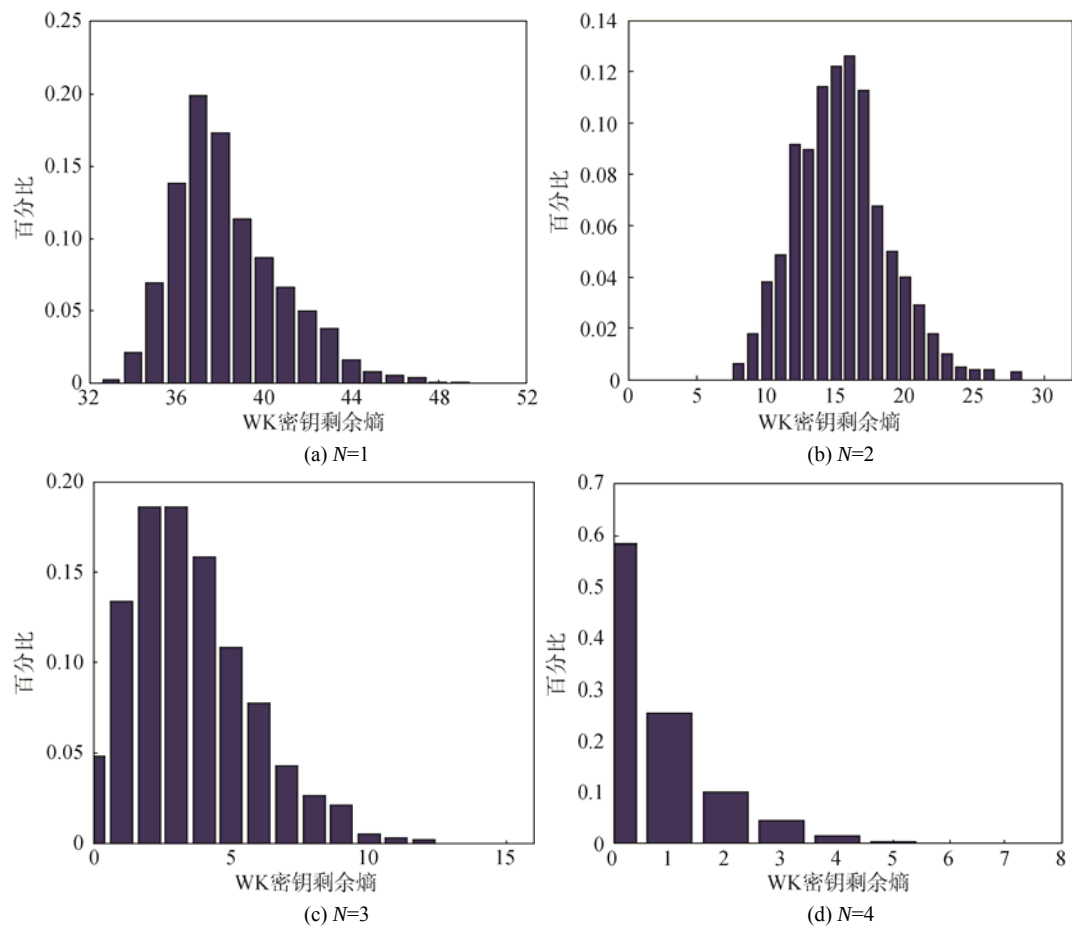


图5 Midori64 最后一轮故障攻击 WK 的密钥剩余熵统计分析
Figure 5 Remaining key entropy of WK after DFA on the last round of Midori64

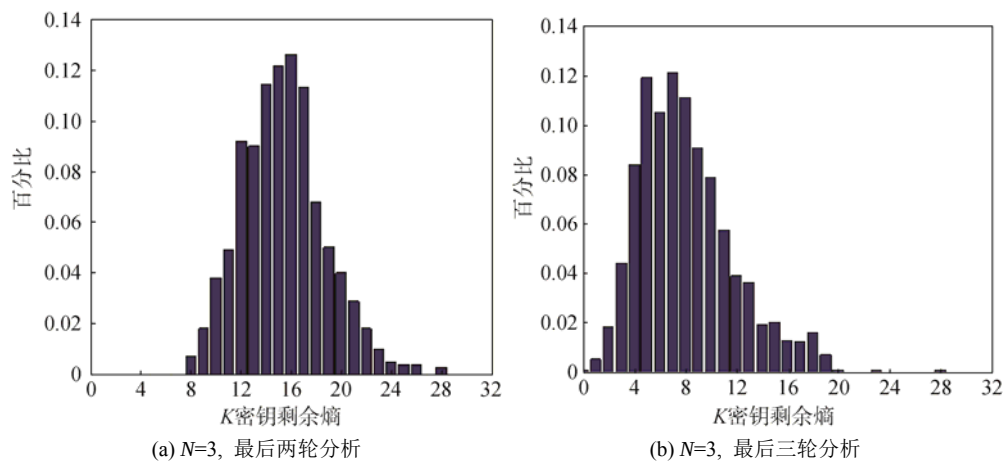


图6 Midori64 最后 2、3 轮故障攻击 K 的密钥剩余熵统计分析
Figure 6 Remaining key entropy of K after DFA on the last two and three rounds of Midori64

在未知故障位置下, 基于第 13 轮注入半字节故障模型, 在不同故障注入次数下执行了攻击实验. 攻击中, 首先通过穷举故障注入位置的可能值组合 16^N , 然后进行故障分析. 实验中, 我们发现 $N=3$ 时, 对于错误的故障注入位置组合, 得到的密钥剩余空间集合为空. 因此, 当 $N=3$ 时, 故障位置未知问题可以等价于故障位置已知问题, 只是需要分析的次数增加为 2^{12} , 攻击可以在 10 分钟内完成.

6.5 Midori128 实验结果与分析

Midori128 算法攻击中, 首先在已知故障位置下, 基于第 17 轮注入字节故障模型, 在不同故障注入次数下执行了攻击实验.

(1) 最后一轮分析

对不同故障次数下最后一轮攻击后 WK 的剩余密钥熵作统计分析, 每次执行 1000 次攻击, 结果见图 7.

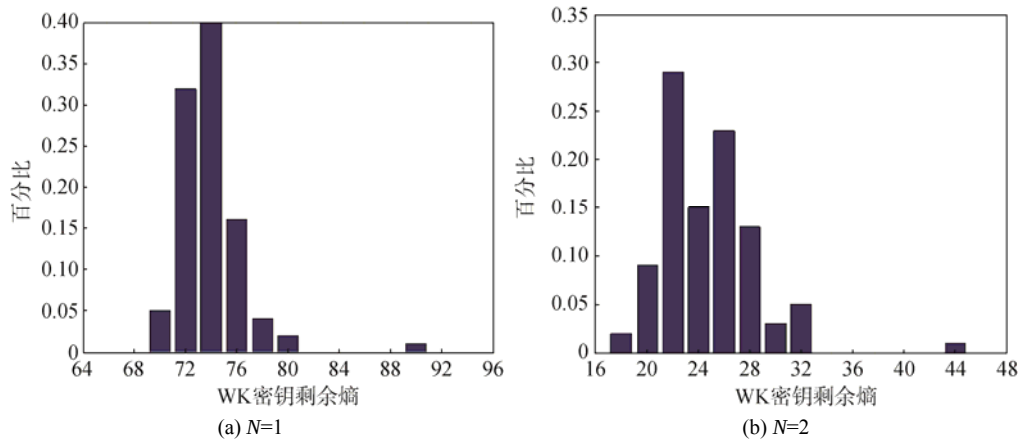


图 7 Midori128 最后一轮故障攻击 WK 的密钥剩余熵统计分析
Figure 7 Remaining key entropy of WK after DFA on the last round of Midori128

可以看出, $N=1$ 时, WK 的密钥剩余熵取值范围为 68 到 92-bit, 均值为 73.96-bit, 接近于理论值 $48 + 3\log_2(255) = 71.98$ -bit; $N=2$ 时, 密钥剩余熵的取值范围为 16 到 45-bit, 均值为 24.69-bit. 大约 2 次故障注入即可将 WK 的搜索空间降低到一个较小的值.

(2) 倒数 2 轮、倒数 3 轮分析

由于 $N=1$ 时, 对倒数第 2、3 轮的计算复杂度过高, 未能在有限时间内恢复密钥. 我们将 N 设定为 2, 通过倒数第 2 轮分析即可恢复唯一的密钥.

此外, 在未知故障位置下, 基于第 17 轮注入字节故障模型, 执行了多次攻击实验. 攻击中, 首先穷举故障注入位置可能值组合 16^N , 然后进行密钥分析. 实验中, $N=2$ 时, 两轮分析即可恢复唯一的主密钥.

7 Midori 算法代数故障分析

7.1 攻击框架

对于 Midori64 和 Midori128 来说, SC 函数的每个输出比特都和 4 个输入比特相关, 本质上还属于 4-bit 的 S 盒, 特别适合于将代数分析同故障分析结合开展代数故障分析. 代数故障分析分为 3 个阶段: 故障注入、故障利用和代数方程求解. 故障利用主要构建故障攻击相关的代数方程, 包括密码方程、故障方程、验证方程. 故障注入部分不是本文重点, 下面重点对故障利用部分进行阐述, 算法 3 给出了 Midori 代数故障分析的伪代码.

算法3 Midori 代数故障分析伪代码

输入: R, N, nmode 输出: $t, \phi(K)$	
$(WK, RK_0, \dots, RK_{R-2}) = KS(K);$	//根据主密钥生成扩展密钥
For ($i = R; i > R - 4; i--$) do	//生成密钥扩展代数方程
GenKSES(K);	
End	
For ($i = 0; i < N; i++$) do	
RandomPT(X_i);	//产生随机明文
$Y_i = \text{Enc}(X_i, K);$	//得到正确加密密文
For ($j = R; j > R - 4; j--$) do	//反向生成最后 4 轮正确加密方程
GenEnRdES();	
End	
GenInputES(Y_i)	//输入正确密文信息
$Y_i^* = \text{InjectFault}(\text{Enc}(X_i, K));$	//注入故障, 得到错误加密密文
For ($j = R; j > R - 4; j--$) do	//反向生成最后 4 轮错误加密方程
GenEnRdES();	
End	
GenInputES(Y_i^*)	//输入错误密文信息
GenFaultES(f)	//将密文差分转化为代数方程
End	
If $\text{nmode} = 0$	//生成随机验证密文并反向构建完整加密方程
RandomPT(X_v);	
$Y_v = \text{Enc}(X_v, K)$	
For ($j = R; j > 0; j--$) do	
GenEnRdES();	
End	
GenInputES(X_v, Y_v)	
(t) = RunAFA();	//得到求解时间
End	
Else	//多个解求解模式
($t, \phi(K)$) = RunAFA();	//得到求解时间和密钥剩余熵
End	

(1) 密码算法验证代数方程组构建

将密码算法表示为关于明文、中间变量、密文、密钥的代数方程组, 在密码代数方程组构造中, 最为关键的部分是如何构造 S 盒、模加、模减等非线性部件以及矩阵乘法等复杂线性部件的代数方程组. 需要说明的是, 由于故障攻击主要利用正确密文和错误密文进行密钥恢复, 为加快解析器方程组求解速度, 在构建加密过程和密钥扩展方程组时, 经验表明从密文开始反向构建解密方程组效果比较好.

(2) 故障注入后几轮方程构建

将故障模型对应的正确和错误加密的最后几轮方程反向构建出来, 具体轮函数构建方法同上.

(3) 故障方程构建

将故障注入差分 and 密文使用代数方程组表示, 同密码算法代数方程组联立, 降低代数方程组的求解复杂度, 加快方程组求解速度.

(4) 代数方程组求解

将故障信息代数方程组同密码算法方程组联立后, 密钥恢复攻击等价于方程组求解问题. 典型的代数

方程组求解主要包括基于可满足性问题^[28](使用 Minisat、CryptoMinisat 等解析器)、基于最优化问题(使用 CPLEX、SCIP 解析器)、基于线性化(直接线性化 XL、扩展线性化 XSL)、基于 Gröbner 基^[29]等方法。

7.2 Midori 算法代数方程构建

(1) SC 逆运算

对于 Midori64 来说, SC 函数由 16 次查找 1 个 4-bit 的 S 盒 Sb_0 进行计算; 对于 Midori128 来说, SC 函数由 32 次查找 1 个 4-bit 的 S 盒 Sb_1 , 并在查表前后分别执行一个基于比特的置换函数组成。需要说明的是, Sb_0 和 Sb_1 都属于自反 S 盒, 逆 S 盒和原始 S 盒相同。令 S 盒输入为 $(x_0 \parallel x_1 \parallel x_2 \parallel x_3)$, 输出为 $(y_0 \parallel y_1 \parallel y_2 \parallel y_3)$, 利用真值表构造法, 可将每个 S 盒输出比特使用所有 S 盒输入比特显式表示出来^[30]。

$$\begin{aligned}
 y_0 &= a_0 + a_1x_0 + a_2x_1 + a_3x_2 + a_4x_3 + a_5x_0x_1 + a_6x_0x_2 + a_7x_0x_3 + a_8x_1x_2 + a_9x_1x_3 + \\
 &\quad a_{10}x_2x_3 + a_{11}x_0x_1x_2 + a_{12}x_0x_1x_3 + a_{13}x_0x_2x_3 + a_{14}x_1x_2x_3 + a_{15}x_0x_1x_2x_3 \\
 y_1 &= a_{16} + a_{17}x_0 + a_{18}x_1 + a_{19}x_2 + a_{20}x_3 + a_{21}x_0x_1 + a_{22}x_0x_2 + a_{23}x_0x_3 + a_{24}x_1x_2 + a_{25}x_1x_3 + \\
 &\quad a_{26}x_2x_3 + a_{27}x_0x_1x_2 + a_{28}x_0x_1x_3 + a_{29}x_0x_2x_3 + a_{30}x_1x_2x_3 + a_{31}x_0x_1x_2x_3 \\
 y_2 &= a_{32} + a_{33}x_0 + a_{34}x_1 + a_{35}x_2 + a_{36}x_3 + a_{37}x_0x_1 + a_{38}x_0x_2 + a_{39}x_0x_3 + a_{40}x_1x_2 + a_{41}x_1x_3 + \\
 &\quad a_{42}x_2x_3 + a_{43}x_0x_1x_2 + a_{44}x_0x_1x_3 + a_{45}x_0x_2x_3 + a_{46}x_1x_2x_3 + a_{47}x_0x_1x_2x_3 \\
 y_3 &= a_{48} + a_{49}x_0 + a_{50}x_1 + a_{51}x_2 + a_{52}x_3 + a_{53}x_0x_1 + a_{54}x_0x_2 + a_{55}x_0x_3 + a_{56}x_1x_2 + a_{57}x_1x_3 + \\
 &\quad a_{58}x_2x_3 + a_{59}x_0x_1x_2 + a_{60}x_0x_1x_3 + a_{61}x_0x_2x_3 + a_{62}x_1x_2x_3 + a_{63}x_0x_1x_2x_3
 \end{aligned} \tag{23}$$

Sb_0 和 Sb_1 对应的 64 比特的变量 a_i 分别如表 2 所示。

表 2 Sb_0 和 Sb_1 对应 64 比特的变量 a_i
Table 2 64-bit variable a_i of Sb_0 and Sb_1

i	Sb_0 对应 a_i	Sb_1 对应 a_i
0–15	1,0,0,0,0,1,1,0,0,0,1,1,0,1,0,0	0,1,1,0,0,1,0,0,0,1,0,1,1,0,0,0
16–31	1,1,0,0,1,0,0,1,0,0,0,1,0,1,1,0	0,1,1,1,0,1,0,1,1,1,0,1,0,0,0,0
32–47	0,0,1,0,1,1,0,1,0,1,0,0,0,0,0,0	0,0,1,0,0,1,0,1,0,0,1,0,1,1,1,0
48–63	0,0,0,1,0,0,0,1,0,1,0,1,0,1,1,0	1,0,1,0,1,0,0,0,1,1,1,1,0,0,1,0

令 Midori128 查找 32 次 S 盒 Sb_1 前后查找的 128 比特置换表用 P1 和 P2 来表示, 根据算法设计, P1 和 P2 表内容如表 3 所示。需要说明的是, P1 和 P2 是互逆的, SC 逆运算等价于先根据 P1 表进行比特置换, 然后查找 32 次 S 盒 Sb_1 , 之后执行 P2 置换。

表 3 Sb_0 和 Sb_1 对应 128 比特的变量 a_i
Table 3 128-bit variable a_i of Sb_0 and Sb_1

i	P1[i]	P2[i]
0–15	4,1,6,3,0,5,2,7,9,14,15,8,13,10,11,12	4,1,6,3,0,5,2,7,11,8,13,14,15,12,9,10
16–31	18,19,20,17,22,23,16,21,31,28,25,26,27,24,29,30	22,19,16,17,18,23,20,21,29,26,27,28,25,30,31,24
32–47	36,33,38,35,32,37,34,39,41,46,47,40,45,42,43,44	36,33,38,35,32,37,34,39,43,40,45,46,47,44,41,42
48–63	50,51,52,49,54,55,48,53,63,60,57,58,59,56,61,62,68,65	54,51,48,49,50,55,52,53,61,58,59,60,57,62,63,56
64–79	70,67,64,69,66,71,73,78,79,72,77,74,75,76	68,65,70,67,64,69,66,71,75,72,77,78,79,76,73,74
80–95	82,83,84,81,86,87,80,85,95,92,89,90,91,88,93,94	86,83,80,81,82,87,84,85,93,90,91,92,89,94,95,88
96–111	100,97,102,99,96,101,98,103,105, 110,111,104,109,106,107,108,	100,97,102,99,96,101,98,103, 107,104,109,110,111,108,105,106
112–127	114,115,116,113,118,119,112,117, 127,124,121,122,123,120,125,126	118,115,112,113,114,119,116,117, 125,122,123,124,121,126,127,120

(2) SFC 逆运算

等价于将 16 个 cell(16b 比特)输入执行一个按 cell 置换函数, 置换向量为

$$R = [0, 7, 14, 9, 5, 2, 11, 12, 15, 8, 1, 6, 10, 13, 4, 3]$$

令输入为 $(x_0 \| x_1 \| \dots \| x_{16b-1})$, 输出为 $(y_0 \| y_1 \| \dots \| y_{16b-1})$, 则 SFC 逆运算可表示为:

$$y_{bi+j} + x_{bR(i)+j} = 0, 0 \leq i < 16, 0 \leq j < 16 \quad (24)$$

(3) MC 逆运算

MC 函数将一个 4×4 的矩阵 M 乘以输入的 16 个 cell, 得到输出. 需要说明的是该函数也是自逆的. 令 MC 函数输入为 $X_0 \| X_1 \| \dots \| X_{15}$, 输出为 $Y_0 \| Y_1 \| \dots \| Y_{15}$, 则该函数可表示为

$$\begin{aligned} Y_{4i} + X_{4i+1} + X_{4i+2} + X_{4i+3} &= 0, Y_{4i+1} + X_{4i+0} + X_{4i+2} + X_{4i+3} = 0 \\ Y_{4i+2} + X_{4i+0} + X_{4i+1} + X_{4i+3} &= 0, Y_{4i+3} + X_{4i+0} + X_{4i+1} + X_{4i+2} = 0 \end{aligned} \quad (25)$$

其中 $0 \leq i < 4$.

(4) AK 逆运算

令 AK 函数逆运算输入为 $(x_0 \| x_1 \| \dots \| x_{16b-1})$, $(y_0 \| y_1 \| \dots \| y_{16b-1})$, 输出为 $(z_0 \| z_1 \| \dots \| z_{16b-1})$, 则 AK 函数可表示为:

$$x_i + y_i + z_i = 0, 0 \leq i < 16b \quad (26)$$

7.3 故障代数方程构建

令 X 表示 16b 比特正确加密状态, 则 $X = (x_0 \| x_1 \| \dots \| x_{16b-1})$; 令 X^* 表示注入宽度为 b 比特故障后的 16b 比特错误加密状态, $X^* = (x_0^* \| x_1^* \| \dots \| x_{16b-1}^*)$, 故障可能位置有 16 个. 令 Z 表示 X 和 X^* 的故障差分,

$$Z = z_0 \| z_1 \| \dots \| z_{16b-1}, Z_i = X_i + X_i^*, 0 \leq i < 16b \quad (27)$$

根据故障宽度大小 w 不同, Z 可以被划分为 16 个 b 比特变量, $Z = (Z_0 \| Z_1 \| \dots \| Z_{15})$

$$Z_i = Z_{b \times i} \| Z_{b \times i + 1} \| \dots \| Z_{b \times i + b - 1}, 0 \leq i < 16 \quad (28)$$

根据攻击者是否已知故障索引值 t , Z 可以用不同的代数方程来表示.

(1) 故障索引 t 已知

假设 t 值已知, 则 Z 可表示为

$$Z_i = 0, 0 \leq i < 16, i \neq t \quad (29)$$

Z_t 是一个非 0 的 16 比特变量, 可通过引入单比特变量 u_t 来表示 Z_t 是否非 0.

$$u_t = (1 \oplus z_{b \times t}) \| (z_{b \times t + 1}) \| \dots \| (1 \oplus z_{b \times t + b - 1}) = 0 \quad (30)$$

根据上式, Z 可通过引入 9 个变量和 $17b+2$ 个 CNF 变量来表示.

(2) 故障索引 t 未知

在实际攻击中, 故障索引 t 可能是未知的, 可通过引入 16 个变量 u_i 来表示 Z_i 是否被注入故障.

$$u_i = (1 \oplus z_{b \times i}) \| (z_{b \times i + 1}) \| \dots \| (1 \oplus z_{b \times i + b - 1}), 0 \leq i < 16 \quad (31)$$

如果 $u_i = 0$, 则 Z_i 即为注入故障的比特. 由于 $u_0, u_1, \dots, u_{15}$ 有且只有一个为 0, 则可表示为

$$(1 - u_0) \vee (1 - u_1) \vee \dots \vee (1 - u_{15}) = 1, u_i \vee u_j = 1, 0 \leq i < j < 16 \quad (32)$$

根据上式, Z 可通过引入 $16(b+2)$ 个变量和 $16(2b+9.5)+1$ 个 CNF 变量来表示.

7.4 Midori64 实验结果与分析

基于第 13 轮注入随机半字节故障模型, 在 $N=3$ 时使用随机明文执行了 100 次攻击实验. 图 8 给出执

行 100 次攻击的密钥剩余熵分布, 可以看出密钥剩余熵遵循指数分布, 和 6.4 节差分故障分析结果基本吻合, 平均可将主密钥 K 降低到 8.25-bit.

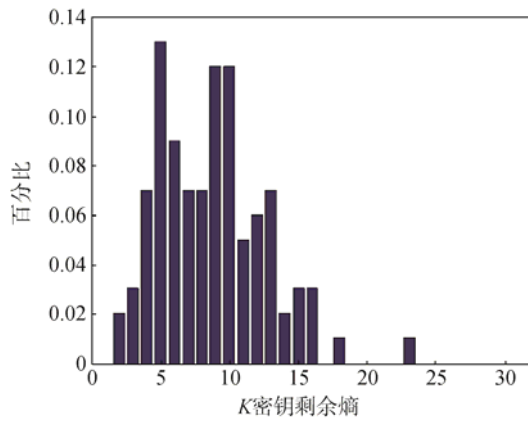


图 8 第 13 轮半字节故障密钥剩余熵分布($N=3$)
Figure 8 Remaining key entropy under 13-th round nibble fault model ($N=3$)

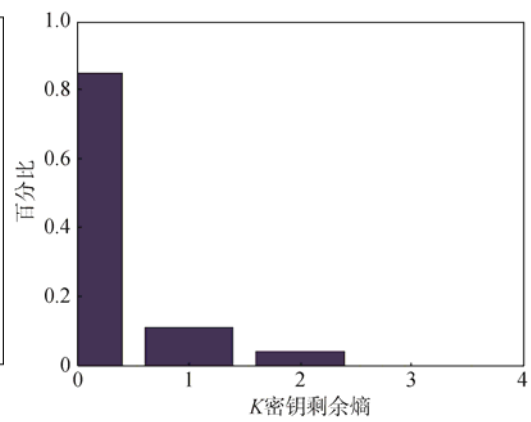


图 9 第 13 轮字节故障密钥剩余熵分布($N=4$)
Figure 9 Remaining key entropy under 13-th round byte fault model ($N=4$)

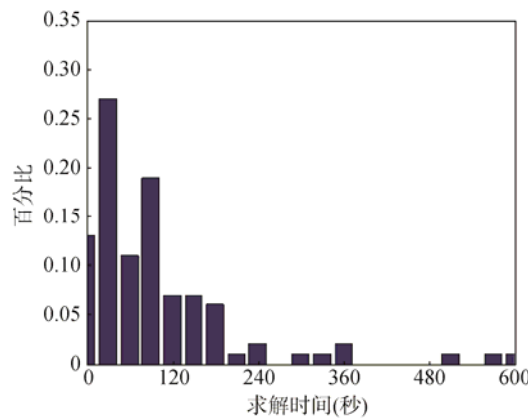


图 10 第 13 轮字节故障求解时间分布($N=4$)
Figure 10 Solving time under 13-th round byte fault model ($N=4$)

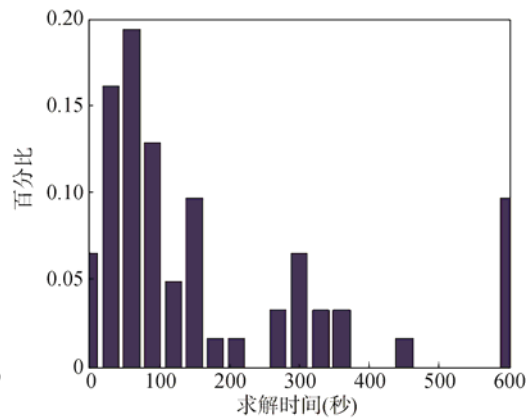


图 11 第 12 轮半字节故障求解时间分布($N=10$)
Figure 11 Solving time under 12-th round byte fault model ($N=10$)

代数故障分析方法可以很容易被扩展到 Midori64 宽度故障模型(如第 13 轮字节故障模型)和深度故障模型(如第 12 轮半字节故障模型), 此时故障在第 15 轮、第 16 轮均会出现故障交错, 此时差分故障分析难以奏效. 但使用代数故障分析方法, 解析器会自动地对故障传播过程进行分析, 相比传统的差分故障分析具有更好的通用性. 仿真实验结果表明, 应用代数故障分析方法, 基于第 13 轮字节故障模型, 4 次故障注入分析将密钥搜索空间降低至 0.19-bit(图 9), 攻击可以 99% 的成功率在 10 分钟内完成(图 10); 基于第 12 轮字节故障模型, 10 次故障注入分析可以 91% 的概率在 10 分钟内恢复主密钥(图 11).

7.5 Midori128 实验结果与分析

基于第 13 轮注入随机字节故障模型, 在 $N=1$ 时使用随机明文执行了 100 次攻击. 图 12 给出执行 100 次攻击的密钥剩余熵分布, 可以看出密钥剩余熵遵循指数分布, 解析器利用 Midori128 的 S 盒方程阶较低

的特性,可以降低差分故障分析的攻击复杂度,1次故障注入可以94%的概率将主密钥降低到16-bit之内.

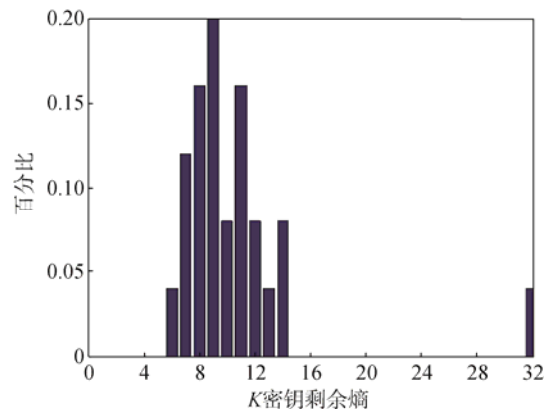


图 12 Midori128 代数故障攻击 K 的密钥剩余熵统计分析
Figure 12 Remaining key entropy in AFA on Midori128

8 结束语

对Midori算法抗故障分析安全性进行了理论分析和仿真实验验证. 基于信息论, 通过分析故障传播路径, 得到每个轮次故障分析大致可恢复的不可能密钥熵, 在此基础上得到主密钥剩余熵的理论值. 利用差分故障分析和代数故障分析方法, 对Midori的两类算法进行了安全性评估. 结果表明: 基于第 $R-3$ 轮半字节故障模型, 3次故障注入可恢复Midori64密钥; 基于第 $R-3$ 轮字节故障模型, 1次故障注入可恢复Midori128主密钥. 因此, 必须至少对Midori算法倒数5轮进行故障攻击防护.

References

- [1] FANG B X. Security of internet of things[J]. Information and Communications Technology, 2010, 4(6): 4. 方滨兴. 关于物联网的安全[J]. 信息通信技术, 2010, 4(6): 4.
- [2] WU C K. An overview on the security techniques and challenges of the internet of things[J]. Journal of Cryptologic Research, 2015, 2(1): 40–53. 武传坤. 物联网安全关键技术与挑战[J]. 密码学报, 2015, 2(1): 40–53.
- [3] BOGDANOV A, KNUDSEN L R, LEANDER G, et al. PRESENT: An ultra-lightweight block cipher[C]. In: Proceedings of CHES 2007. Springer Berlin Heidelberg, 2007: 450–466.
- [4] IZADI M, SADEGHIYAN B, SADEGHIAN S S, et al. MIBS: A new lightweight block cipher[C]. In: Proceedings of CANS 2009. Springer Berlin Heidelberg, 2009: 334–348.
- [5] Dolmatov V. RFC 5830: GOST 28147-89 encryption, decryption and MAC algorithms[R/OL]. IETF (March 2010), ISSN: 2070-1721. <http://tools.ietf.org/html/rfc5830>, 2010.
- [6] GONG Z, NIKOVA S, LAW Y W. KLEIN: a new family of lightweight block ciphers[C]. In: International Workshop on Radio Frequency Identification: Security and Privacy Issues. Springer Berlin Heidelberg, 2011: 1–18.
- [7] GUO J, PEYRIN T, POSCHMANN A, et al. The LED block cipher[C]. In: Proceedings of CHES 2011. Springer Berlin Heidelberg, 2011: 326–341.
- [8] SHIBUTANI K, ISOBE T, HIWATARI H, et al. Piccolo: An ultra-lightweight block cipher[C]. In: Proceedings of CHES 2011, Springer Berlin Heidelberg, 2011: 342–357.
- [9] WU W, ZHANG L. LBlock: a lightweight block cipher[C]. In: International Conference on Applied Cryptography and Network Security. Springer Berlin Heidelberg, 2011: 327–344.
- [10] Borghoff J, Canteaut A, Güneysu T, et al. PRINCE—a low-latency block cipher for pervasive computing applications[C]. In: International Conference on the Theory and Application of Cryptology and Information Security. Springer Berlin Heidelberg, 2012: 208–225.
- [11] BANIK S, BOGDANOV A, ISOBE T, et al. Midori: A block cipher for low energy[C]. In: Proceedings of ASIACRYPT 2015, Springer Berlin Heidelberg, 2015: 411–436.

- [12] BONEH D, DEMILLO R A, LIPTON R J. On the importance of checking cryptographic protocols for faults[C]. In: International Conference on the Theory and Applications of Cryptographic Techniques. Springer Berlin Heidelberg, 1997: 37–51.
- [13] BIHAM E, SHAMIR A. Differential Fault Analysis of Secret Key Cryptosystems[C]. In: Proceedings of the 17th Annual International Cryptology Conference on Advances in Cryptology. Springer Berlin Heidelberg, 1997: 513–525.
- [14] HEMME L. A differential fault attack against early rounds of (Triple-)DES[C]. In: Proceedings of CHES 2004. Springer Berlin Heidelberg, 2004: 254–267.
- [15] PIRET G, QUISQUATER J J. A differential fault attack technique against spn structures, with application to the AES and Khazad[C]. In: Proceedings of CHES 2003. Springer Berlin Heidelberg, 2003: 77–88.
- [16] LI W, GU D W, LI J R. Differential fault analysis on the ARIA algorithm[J]. Information Sciences, 2008: 3727–3737.
- [17] ZHOU Y B, WU W L, XU N N, et al. Differential fault attack on Camellia[J]. Chinese Journal of Electronics, 2009, 18(1): 13–19.
- [18] ZHANF L, WU W L. Differential fault analysis on SMS4[J]. Chinese Journal of Computers, 2006, 29(9): 1596–1602.
张蕾, 吴文玲. SMS4 密码算法的差分故障攻击[J]. 计算机学报, 2006, 29(9): 1596–1602.
- [19] COURTOIS N, WARE D, JACKSON K. Fault-algebraic attacks on inner rounds of DES[C]. In: Proceedings of eSmart 2010. 2010: 22–24.
- [20] JOVANOVIĆ P, KREUZER M, POLIAN I. An algebraic fault attack on the LED block cipher[J]. IACR Cryptology ePrint Archive, 2012, 2012: 400.
- [21] ZHANG F, ZHAO X J, GUO S Z, et al. Improved algebraic fault analysis: A case study on Piccolo and applications to other lightweight block ciphers[C]. In: Proceedings of COSADE 2013. Springer Berlin Heidelberg, 2013: 62–79.
- [22] LIN L, WU W. Meet-in-the-middle attacks on reduced-round Midori-64[J]. IACR Cryptology ePrint Archive, 2015, 2015: 1165.
- [23] GUO J, JEAN J, NIKOLIC I, et al. Invariant subspace attack against full Midori64[J]. IACR Cryptology ePrint Archive, 2015, 2015: 1189.
- [24] CHEN Z, WANG X. Impossible differential cryptanalysis of Midori[J]. IACR Cryptology ePrint Archive, 2016, 2016: 535.
- [25] TODO Y, LEANDER G, SASAKI Y. Nonlinear invariant attack: Practical attack on full SCREAM, i SCREAM, and Midori 64[C]. In: Advances in Cryptology—ASIACRYPT 2016. Springer Berlin Heidelberg, 2016: 3–33.
- [26] BAR-EL H, CHOUKRI H, NACCACHE D, et al. The sorcerer's apprentice guide to fault attacks[J]. Proceedings of the IEEE, 2006, 94(2): 370–382.
- [27] FUKUNAGA T, TAKAHASHI J. Practical fault attack on a cryptographic LSI with ISO/IEC 18033-3 block ciphers[C]. In: Proceedings of FDTC2009. Springer Berlin Heidelberg, 2009: 84–92.
- [28] SOOS M, NOHL K, CASTELLUCCIA C. Extending SAT solvers to cryptographic problems[C]. In: Proceedings of SAT 2009. Springer Berlin Heidelberg, 2009: 244–257.
- [29] Faugère J C. Gröbner bases: Applications in cryptology[R/OL]. In: FSE 2007. <http://fse2007.uni.lu/slides/faugere.pdf>.
- [30] KNUDSEN L R, MIOLANE C V. Counting equations in algebraic attacks on block ciphers[J]. International Journal of Information Security, 2010, 9(2): 127–135.

作者信息



王艺迪(1992–), 辽宁营口人, 硕士研究生. 主要研究领域为对称密码故障分析.
E-mail: wangyidiem@sina.com



赵新杰(1986–), 河南开封人, 博士, 工程师. 主要研究领域为网络空间安全与密码学.
E-mail: zhaoxinjieem@163.com



张帆(1978–), 浙江杭州人, 博士, 副教授. 主要研究领域为网络空间安全、系统安全和密码学.
E-mail: fanzhang@zju.edu.cn



郭世泽(1969–), 河北石家庄人, 博士, 研究员. 主要研究领域为网络空间安全.
E-mail: nsfgsz@126.com



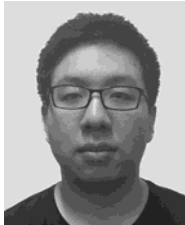
吴礼发(1968-), 湖北蕲春人, 博士, 教授. 主要研究领域为网络空间安全.

E-mail: wulifa@vip.163.com



李文(1989-), 四川绵阳人, 硕士研究生, 助理工程师. 主要研究领域为分组密码故障分析.

E-mail: lianwen.cool@163.com



楼潇轩(1995-), 浙江金华人. 浙江大学信息与电子工程学院本科生.

E-mail: xiaoxuanlou@zju.edu.cn

中国密码学会 2017 年会征文通知

中国密码学会 2017 年会(ChinaCrypt2017)定于 2017 年 10 月 27 日至 29 日在泉城济南举行. 本次年会由中国密码学会主办, 山东大学承办, 旨在汇聚国内密码学领域中从事学术理论研究和应用技术开发的专家学者、行业精英、工程技术人员和在校研究生, 共同探讨密码学领域各方向的最新成果、学术热点、研究动态及发展趋势, 促进密码学术界和产业界的相互交流与合作, 推动我国密码理论与技术的发展.

会议现面向全国从事密码学 and 信息安全领域研究的专家学者、科研工作者、工程技术人员以及在校研究生公开征稿.

一、征文内容

年会征文涵盖密码学理论和应用的各个分支. 征文范围包括但不限于: 对称密码、公钥密码、数字签名、杂凑函数、安全协议、密钥管理、后量子密码算法、量子密码学、生物特征密码技术、侧信道分析与泄露容忍密码算法、密码芯片、密码软件、大数据安全、区块链技术与应用、云计算安全、物联网与智能终端安全以及工控安全等. 会议将编辑年会论文集作为会议交流资料(不正式出版, 允许再投稿), 投稿论文可以是已向重要学术刊物或国际会议投稿或新近发表的高水平研究成果.

二、重要日期

论文投稿截止时间: 2017 年 7 月 31 日; 论文录用通知时间: 2017 年 9 月 29 日

三、投稿要求

1. 通过 E-mail 发送电子稿, 联系和投稿邮箱为: cacr2017@cacnet.org.cn.
2. 来稿内容应是作者本人的科研成果, 数据真实、可靠, 具有较高的学术价值或应用价值.
3. 论文用 word 格式排版. 另附论文标题、作者、单位、E-mail 地址、通信地址、电话等信息.
4. 论文编排顺序如下: (1)标题; (2)作者; (3)单位; (4)中文摘要、关键词; (5)英文摘要、关键词; (6)正文; (7)参考文献.

四、联系方式

稿件联系人: 山东大学计算机科学与技术学院 蒋瀚(18660749850), 胡程瑜(13853142022)

中国密码学会

2017 年 2 月 7 日